

DTE Platform — Solution Bootstrap

Generation Specification

As Lead Engineer, I am issuing the final implementation blueprint. This document defines the exact physical state of the repository at **Commit 0**.

There is no more architecture to discuss. There are no more technology choices to make. This is the explicit, deterministic specification for generating the source code skeleton.

1. Commit 0 Objectives

What explicitly EXISTS after Commit 0:

- The complete directory structure.
- The DtePlatform.sln solution file.
- All .csproj files with Central Package Management configured.
- Global compiler enforcement files (Directory.Build.props, .editorconfig).
- Docker compose files for local infrastructure (PostgreSQL).
- CI/CD GitHub Action workflows (failing if architecture rules are broken).
- DTE.Architecture.Tests asserting layer boundaries.
- Empty Program.cs, ApplicationDbContext, and MediatR registration stubs.

What explicitly DOES NOT EXIST after Commit 0:

- Domain Aggregates (e.g., FiscalDocument.cs).
- API Controllers.
- EF Core Migrations (Migration 000 comes in Commit 2).
- Business logic of any kind.

2. Physical Repository Tree

The repository generator must produce this exact file tree:

Plaintext

```
|
|
|— github
|   |
|   |— workflow
|       |
|       |— dym
|           |
|           |— pr-validation.yml
|               |
|               |— architecture-validation.yml
|                   |
|                   |— build
|                       |
|                       |— Directory.Build.props
|                           |
|                           |— Directory.Packages.props
```

```
├── deploy
├── docker
│   ├── Dockerfile
│   ├── docker-compose.dev.yml
│   └── docker-compose.prod.yml
├── docs
│   ├── architecture
│   │   └── decisions
│   │       └── 0001-record-architecture-decisions.md
│   └── onboarding
│       ├── local-setup.md
│       └── README.md
├── scripts
│   ├── db-init.sql
│   └── run-local.ps1
├── src
│   ├── DTE.Domain
│   │   ├── DTE.Domain.csproj
│   │   └── AssemblyReference.cs
│   ├── DTE.Application
│   │   ├── DTE.Application.csproj
│   │   └── AssemblyReference.cs
│   ├── DTE.Infrastructure
│   │   ├── DTE.Infrastructure.csproj
│   │   └── AssemblyReference.cs
│   ├── DTE.Api
│   │   ├── DTE.Api.csproj
│   │   ├── Program.cs
│   │   ├── appsettings.json
│   │   └── appsettings.Development.json
│   └── tests
│       ├── DTE.Domain.Tests
│       │   ├── DTE.Domain.Tests.csproj
│       │   └── DTE.Application.Tests
│       ├── DTE.Application.Tests.csproj
│       ├── DTE.Infrastructure.Tests
│       │   ├── DTE.Infrastructure.Tests.csproj
│       │   ├── DTE.Api.Tests
│       │   ├── DTE.Api.Tests.csproj
│       │   ├── DTE.Architecture.Tests
│       │   └── DTE.Architecture.Tests.csproj
│       └── Layer.Tests.cs
└── editorconfig
```



3. Solution Definition

DtePlatform.sln Configuration:

- **Format:** Visual Studio 2022 Solution File (Format Version 12.00).
- **Project GUID Strategy:** Use deterministic, standard UUIDv4 generated at creation.
- **Folder Mapping:**
 - src/ $\rightarrow$ DTE.Domain, DTE.Application, DTE.Infrastructure, DTE.Api
 - tests/ $\rightarrow$ DTE.Domain.Tests, DTE.Application.Tests, DTE.Infrastructure.Tests, DTE.Api.Tests, DTE.Architecture.Tests
 - Solution Items $\rightarrow$.editorconfig, Directory.Build.props, Directory.Packages.props, global.json, README.md

Project Reference Matrix (Enforced by compiler):

- DTE.Api references: DTE.Application, DTE.Infrastructure
- DTE.Infrastructure references: DTE.Application, DTE.Domain
- DTE.Application references: DTE.Domain
- DTE.Domain references: *None*
- DTE.Architecture.Tests references: *All src projects*

4. Project Bootstrap Files

All .csproj files share these inherited properties (via Directory.Build.props), but must explicitly define their package references via Central Package Management (CPM).

DTE.Domain.csproj

- Target: net8.0
- Packages: *None*.

DTE.Application.csproj

- Target: net8.0
- Packages: MediatR, FluentValidation, DependencyInjectionExtensions.

DTE.Infrastructure.csproj

- Target: net8.0
- Packages: Microsoft.EntityFrameworkCore, Npgsql.EntityFrameworkCore.PostgreSQL, Hangfire.PostgreSql, Hangfire.AspNetCore.

DTE.Api.csproj

- Target: net8.0
- OutputType: Exe
- Packages: Serilog.AspNetCore, Microsoft.AspNetCore.Authentication.JwtBearer, Microsoft.EntityFrameworkCore.Design.

5. Global Repository Files

global.json

- Pins SDK to exact version 8.0.x to prevent CI/CD drift.

build/Directory.Build.props

- TargetFramework: net8.0
- ImplicitUsings: enable
- Nullable: enable
- TreatWarningsAsErrors: true (Mandatory: Code does not compile with warnings).
- EnforceCodeStyleInBuild: true

build/Directory.Packages.props

- ManagePackageVersionsCentrally: true
- Defines exact versions for Npgsql, EF Core, Hangfire, MediatR, Testcontainers, xUnit, NetArchTest. (Ensures all projects use identical library versions).

.editorconfig

- Enforces: csharp_style_namespace_declarations = file_scoped
- Enforces: K&R brace style, 4-space indentation.
- Enforces: Naming conventions (Interfaces start with I, Private fields start with _).

6. Development Environment Bootstrap

Required Tooling:

- dotnet-sdk-8.0
- Docker Desktop or compatible daemon.
- EF Core CLI: dotnet tool install --global dotnet-ef

Local Developer Commands (Mapped in run-local.ps1):

- .\scripts\run-local.ps1 -Command up: Boots docker-compose DB.
- .\scripts\run-local.ps1 -Command migrate: Runs EF migrations.
- .\scripts\run-local.ps1 -Command test: Runs full test suite.

Environment Variable Conventions:

- Local secrets managed via: dotnet user-secrets init --project src/DTE.Api
- Prefix all platform variables with DTE__.

7. Docker Bootstrap

deploy/docker/Dockerfile

- **Stage 1 (Build):** mcr.microsoft.com/dotnet/sdk:8.0. Runs dotnet publish -c Release -o /app.
- **Stage 2 (Runtime):** mcr.microsoft.com/dotnet/aspnet:8.0-alpine.
- **Security:** USER \$APP_UID (Runs as non-root). Exposes port 8080.

deploy/docker-compose.dev.yml

- **Service postgres:** Image postgres:15-alpine.
- **Ports:** 5432:5432.
- **Environment:** POSTGRES_USER=dte_user, POSTGRES_PASSWORD=dte_dev_pass, POSTGRES_DB=dte_platform.
- **Volumes:** Mounts ./scripts/db-init.sql:/docker-entrypoint-initdb.d/init.sql.

8. GitHub Bootstrap

All workflows execute on ubuntu-latest.

.github/workflows/architecture-validation.yml

- Trigger: On Push.
- Action: dotnet test tests/DTE.Architecture.Tests/. Fails immediately if boundaries are breached.

.github/workflows/ci.yml

- Execution Order:
 1. dotnet format --verify-no-changes (Fails if .editorconfig violated).
 2. dotnet build --no-restore -c Release (Fails on warnings).
 3. dotnet test tests/DTE.Domain.Tests
 4. dotnet test tests/DTE.Application.Tests
 5. dotnet test tests/DTE.Infrastructure.Tests (Requires Docker host for Testcontainers).

9. Documentation Bootstrap

ADR Structure (docs/architecture/decisions/):

- Format: MADR (Markdown Architectural Decision Records).
- Status: Accepted / Rejected / Superseded.
- Context, Decision, Consequences.

Onboarding (docs/onboarding/local-setup.md):

- Exactly 5 copy-pasteable terminal commands to go from git clone to a running API returning HTTP 200 on /health.

10. Test Bootstrap

Test Folders & Naming:

- Mirrors source exactly (e.g., tests/DTE.Domain.Tests/Issuance/FiscalDocumentTests.cs).
- Class Naming: [ClassName]Tests.
- Method Naming: [MethodName]_[Scenario]_[ExpectedResult] (e.g., Validate_WhenTotalsMismatch_ThrowsDomainException).

DTE.Architecture.Tests/LayerTests.cs Bootstrap:

- Requires exact NetArchTest rule:
Types.InAssembly(DomainAssembly).ShouldNot().HaveDependencyOn("DTE.Infrastructure").GetResult().IsSuccessful

11. Database Bootstrap

- **Migration 000:** Will be generated post Commit 0. The DbContext is stubbed as ApplicationDbContext in DTE.Infrastructure.Persistence.
- **Schema Ownership:** All tables default to the public schema in PostgreSQL.
- **MH Catalog Ownership:** Handled via EF Core HasData seeding in ApplicationDbContext.OnModelCreating (stubbed in Commit 0).

12. Security Bootstrap

- **Secrets Strategy:** appsettings.json contains blank placeholders ("").

appsettings.Development.json contains local dev passwords (e.g., dte_dev_pass).

- **Certificate Strategy:** ICertificateVault interface added to DTE.Application.Security.
- **Local Development Bootstrap:** LocalCertificateVault stub added to DTE.Infrastructure.Security, configured to read dummy bytes.

13. First Walking Skeleton (Class Inventory)

Commit 0 creates the repository. Commit 1 & 2 will create these exact files to prove the architecture:

Domain (Commit 1):

- DTE.Domain/Issuance/Aggregates/FiscalDocument.cs
- DTE.Domain/Issuance/ValueObjects/DteType.cs
- DTE.Domain/Issuance/ValueObjects/ControlNumber.cs

Application (Commit 1):

- DTE.Application/Issuance/Commands/CreateDraft/CreateDraftCommand.cs
- DTE.Application/Issuance/Commands/CreateDraft/CreateDraftCommandHandler.cs
- DTE.Application/Issuance/Commands/CreateDraft/CreateDraftCommandValidator.cs

Infrastructure (Commit 2):

- DTE.Infrastructure/Persistence/ApplicationDbContext.cs
- DTE.Infrastructure/Persistence/Configurations/FiscalDocumentConfiguration.cs

Api (Commit 2):

- DTE.Api/Controllers/V1/DocumentsController.cs
- DTE.Api/Contracts/Requests/CreateDraftRequest.cs
- DTE.Api/Contracts/Responses/DocumentResponse.cs

14. Commit Sequence

To ensure systematic validation, code generation must follow this strict sequence:

- **Commit 0:** Repository Bootstrap (The output of this specification). CI passes. Arch tests pass.
- **Commit 1:** Core Domain & CQRS stubs for CreateDraft. Unit tests pass.
- **Commit 2:** EF Core Configurations & Database Migration 000. Testcontainers integration tests pass.
- **Commit 3:** API Controllers & JWT Middleware setup. End-to-End tests pass.
- **Commit 4:** Hangfire configuration & SignDocument cryptographic stubs. Evidence BYTEA mapping complete.

15. Repository Generation Readiness Checklist

Before executing the generation script or allowing an engineer to push Commit 0, verify:

- [] Does DtePlatform.sln contain exactly 5 source projects and 5 test projects?
- [] Is Directory.Build.props enforcing TreatWarningsAsErrors=true?
- [] Is NetArchTest.Rules referenced in the Architecture test project?
- [] Are all EF Core references strictly confined to DTE.Infrastructure and DTE.Api (for tools)?
- [] Is docker-compose.dev.yml configured to use PostgreSQL 15?
- [] Does the GitHub Actions YAML contain the dotnet format --verify-no-changes step?

Approval: If the checklist is green, proceed to generation. The repository is ready.